CSA1737 – ARTIFICIAL INTELLIGENCE

LIST OF EXPERIMENTS

6. Write a Python program to implement List methods (Add, Append, Extend & Delete).

AIM:

To write a Python program to implement list methods such as add, append, extend and delete.

ALGORITHM:

1. Start the program.
2. Create a list.
3. Add elements using append() and extend().
4. Insert an element using insert().
5. Delete elements using remove(), pop() and del.
6. Display the results.
7. Stop the program.

CODE:

```
my_list = [10, 20, 30]
print("Original List:", my_list)
my_list.append(40)
print("After Append:", my_list)
my_list.extend([50, 60])
print("After Extend:", my_list)
my_list.insert(1, 15)
print("After Insert:", my_list)
my_list.remove(30)
print("After Remove:", my_list)
my_list.pop()
print("After Pop:", my_list)
del my_list[0]
print("After Delete:", my_list)
```

OUTPUT:

```
Original List: [10, 20, 30]
After Append: [10, 20, 30, 40]
After Extend: [10, 20, 30, 40, 50, 60]
After Insert: [10, 15, 20, 30, 40, 50, 60]
After Remove: [10, 15, 20, 40, 50, 60]
After Pop: [10, 15, 20, 40, 50]
After Delete: [15, 20, 40, 50]
```

7. Write a Python program to Illustrate Different Set Operations.

AIM:

To write a Python program to perform different set operations.

ALGORITHM:

1. Start the program.
2. Initialize two sets, A and B.
3. Perform union, intersection and difference operations.
4. Perform symmetric difference.
5. Display the results.
6. Stop the program.

CODE:

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
print("Set A:", A)
print("Set B:", B)
print("Union:", A | B)
print("Intersection:", A & B)
print("Difference A-B:", A - B)
print("Difference B-A:", B - A)
print("Symmetric Difference:", A ^ B)
```

OUTPUT:

```
Set A: {1, 2, 3, 4}
Set B: {3, 4, 5, 6}
Union: {1, 2, 3, 4, 5, 6}
Intersection: {3, 4}
Difference A-B: {1, 2}
Difference B-A: {5, 6}
Symmetric Difference: {1, 2, 5, 6}
```

8. Write a Python program to generate Calendar for the given month and year.

AIM:

To write a Python program to generate a calendar for a given month and year.

ALGORITHM:

1. Start the program.
2. Import the calendar module.
3. Get the month and year from the user.
4. Generate the calendar using the month() function.
5. Display the calendar.
6. Stop the program.

CODE:

```
1  import calendar
2  year = int(input("Enter year: "))
3  month = int(input("Enter month (1-12): "))
4  print(calendar.month(year, month))
```

OUTPUT:

```
Enter year: Enter month (1-12):    October 2026
Mo Tu We Th Fr Sa Su
        1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30 31
```

9. Write a Python program to remove punctuations from the given string.

AIM:

To write a Python program to remove punctuation marks from a given string.

ALGORITHM:

1. Start the program.
2. Import the string module.
3. Get a string from the user.
4. Initialize an empty string.
5. Iterate through each character.
6. Add only non-punctuation characters to the result.
7. Display the result.
8. Stop the program.

CODE:

```
import string
text = input("Enter a string: ")
result = ""
for ch in text:
    if ch not in string.punctuation:
        result += ch
print("String without punctuation:", result)
```

OUTPUT:

Enter a string: Hello, World! How are you?

```
String without punctuation:  Hello World How are you
```

10. Write a Python program to solve 8-Puzzle problem.

AIM:

To write a Python program to solve the 8-Puzzle problem using Breadth-First Search (BFS).

ALGORITHM:

1. Start the program.
2. Initialize the initial state and goal state.
3. Create a queue and insert the initial state.
4. Remove a state from the queue.
5. Check whether the current state matches the goal state.
6. Generate new states by moving the blank tile up, down, left or right.
7. Add unvisited states to the queue.
8. Repeat until the goal state is reached or the queue becomes empty.
9. Display the solution path.
10. Stop the program.

CODE:

```
from collections import deque
start = (1, 2, 3,
         4, 0, 6,
         7, 5, 8)
goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)
def bfs(start, goal):
    queue = deque([(start, [start])])
    visited = {start}
    moves = [(-1, 0), (1, 0),
             (0, -1), (0, 1)]
    while queue:
        state, path = queue.popleft()

        if state == goal:
            return path
        zero = state.index(0)
        row, col = divmod(zero, 3)
        for dr, dc in moves:
            nr = row + dr
            nc = col + dc
            if 0 <= nr < 3 and 0 <= nc < 3:
                new_pos = nr * 3 + nc
                new_state = list(state)
                new_state[zero], new_state[new_pos] = (
                    new_state[new_pos], new_state[zero]
                )
                new_state = tuple(new_state)
                if new_state not in visited:
                    visited.add(new_state)
                    queue.append(
                        (new_state, path + [new_state])
                    )
    return None
solution = bfs(start, goal)
if solution:
    print("Solution found:")
    for state in solution:
        for i in range(0, 9, 3):
            print(state[i:i + 3])
        print()
    print("Total moves:", len(solution) - 1)
else:
    print("No solution exists")
```

OUTPUT:

```
Solution found:
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)

Total moves: 2
```